

SIMATS ENGINEERING

ASSESSMENT TOOL 1

Experiments

COURSE NAME: Computer Networks

COURSE CODE: CSA07

COURSE OUTCOME COVERED

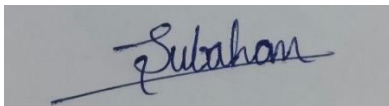
CO3: *Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions*

Assessment Tool Weightage: 40%

Code of Conduct:

I T.SUBAHAN(192511121) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student

A rectangular box containing a handwritten signature in blue ink. The signature appears to be 'Subahan' with a stylized flourish at the end.

Experiments

QUESTIONS:4

Total Marks:40

1. Capture packets during a TCP connection setup. Identify the three packets involved in the handshake (SYN, SYN-ACK, ACK). Demonstrate the role of each flag and how they establish a reliable connection. (BL4) (10)
2. Capture a DNS query using UDP in Wireshark. Compare the UDP header size with TCP and note the absence of sequence numbers or acknowledgments. Discuss how this lightweight design impacts speed and reliability. (BL4) (10)
3. Simulate packet loss (e.g., disconnect briefly) and capture traffic in Wireshark. Observe how TCP retransmits lost packets while UDP does not. Explain why this difference makes TCP reliable but slower compared to UDP. (BL4) (10)
4. Use Wireshark to capture DNS traffic while resolving a domain name. Identify the query type (A, AAAA, MX) and the response received. Measure the time taken for resolution and explain why DNS typically uses UDP.(BL4) (10)

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Experiment 1: TCP Three-Way Handshake

Aim

To capture packets during TCP connection establishment using Wireshark and identify the SYN, SYN-ACK, and ACK packets involved in the three-way handshake.

Requirements

- Computer/Laptop
- Wireshark
- Internet connection
- Web browser

Procedure

1. Open Wireshark.
2. Select the active network interface such as Wi-Fi or Ethernet.
3. Start packet capture.
4. Open a web browser and visit any website.
5. Stop the capture after the website loads.
6. Apply the filter:

`tcp.flags.syn == 1`

7. Identify the TCP packets involved in connection establishment.
8. Observe the following three packets:

- SYN
- SYN-ACK
- ACK

Observation

Packet	Flags	Purpose
1	SYN	Client requests a TCP connection
2	SYN + ACK	Server accepts the request and acknowledges it
3	ACK	Client acknowledges the server's response

Working of Three-Way Handshake

Step 1 – SYN

The client sends a SYN packet to the server.

Client	Server
----- SYN ----->	

The SYN flag indicates that the client wants to establish a TCP connection. It also contains an initial sequence number.

Step 2 – SYN-ACK

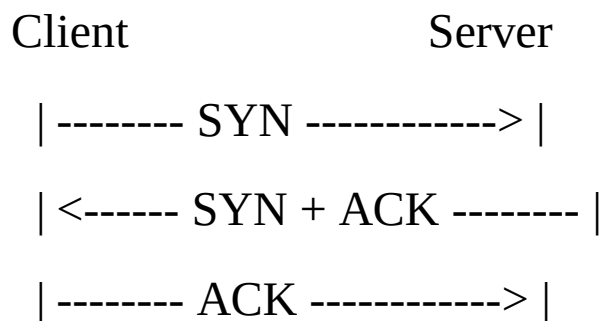
The server responds with SYN + ACK.

Client	Server
----- SYN ----->	
<----- SYN + ACK -----	

- SYN indicates that the server also wants to establish the connection.
- ACK acknowledges the client's SYN.
- The server sends its own initial sequence number.

Step 3 – ACK

The client sends an ACK.



The ACK confirms that the client received the server's SYN-ACK.

After this, the TCP connection is established and data transmission can begin.

Role of Flags

- SYN: Synchronizes sequence numbers and starts a TCP connection.
- ACK: Confirms successful receipt of data/control information.
- SYN + ACK: Server accepts the connection request and acknowledges the client's SYN.

Result

The TCP three-way handshake was successfully captured in Wireshark. The SYN, SYN-ACK, and ACK packets establish

a reliable connection by synchronizing sequence numbers and confirming communication between the client and server.

Experiment 2: DNS Query Using UDP

Aim

To capture a DNS query using UDP in Wireshark and compare the UDP header with the TCP header, particularly the absence of sequence numbers and acknowledgments.

Requirements

- Computer/Laptop
- Wireshark
- Internet connection
- Command Prompt/Terminal

Procedure

1. Open Wireshark.
2. Select the active network interface.
3. Start packet capture.
4. Open Command Prompt.
5. Enter:

```
nslookup google.com
```

6. Stop Wireshark capture.
7. Apply the filter:

dns

or

udp.port == 53

8. Select a DNS query packet.
9. Expand the User Datagram Protocol section.
10. Observe the UDP header fields.

UDP Header

The UDP header contains only 8 bytes.

It consists of:

- Source Port
- Destination Port
- Length
- Checksum

TCP Header

The TCP header is normally 20 bytes or more because it contains additional fields such as:

- Source Port
- Destination Port
- Sequence Number
- Acknowledgment Number
- Flags

- Window Size
- Checksum
- Urgent Pointer
- Options

Comparison

Feature	UDP	TCP
Header size	8 bytes	Minimum 20 bytes
Sequence number	No	Yes
Acknowledgment number	No	Yes
Connection establishment	No	Yes
Retransmission	No	Yes
Reliability	Low	High
Speed	Faster	Relatively slower
DNS usage	Commonly used	Used when required

Why UDP is Faster

UDP does not establish a connection before sending data. It also does not wait for acknowledgments or retransmit lost packets.

For DNS, the client can simply send:

DNS Query → DNS Server

and receive:

DNS Response ← DNS Server

This reduces overhead and delay.

Effect on Reliability

Since UDP does not provide:

- Acknowledgments
- Sequence numbers
- Automatic retransmission

a lost DNS packet is not recovered by UDP itself. The DNS application can handle the situation by sending another query.

Result

The DNS query was successfully captured using UDP. UDP has a small 8-byte header and does not contain TCP-style sequence and acknowledgment numbers. This lightweight design provides faster communication with less overhead but offers less transport-level reliability.

Experiment 3: TCP Retransmission vs UDP

Aim

To simulate packet loss and observe how TCP retransmits lost packets while UDP does not provide transport-level retransmission.

Requirements

- Computer/Laptop
- Wireshark
- Internet connection
- Command Prompt/Terminal

Procedure

1. Open Wireshark.
2. Select the active network interface.
3. Start packet capture.
4. Generate TCP traffic by opening a website or downloading a file.
5. To observe TCP retransmission, temporarily interrupt or disturb the network connection during active TCP communication.
6. Restore the connection.
7. Stop the capture.
8. Apply the filter:

`tcp.analysis.retransmission`

9. Observe packets marked as TCP Retransmission.

TCP Packet Loss

Suppose a TCP packet is lost:

Sender

Receiver

| ---- Packet 1 -----> |

| ---- Packet 2 -----> | X Lost

| ---- Packet 3 -----> |

The receiver detects missing data using sequence numbers and acknowledgments.

TCP eventually retransmits the missing packet:

Sender Receiver

| ---- Packet 2 -----> |

Therefore, TCP ensures that lost data can be recovered.

TCP Reliability Mechanisms

TCP uses:

- Sequence numbers
- Acknowledgments
- Retransmission
- Timeout mechanisms
- Flow control
- Congestion control

These mechanisms make TCP reliable.

UDP Packet Loss

UDP does not have TCP-style retransmission.

Sender

Receiver

| ---- UDP Packet -----> | X Lost

If the packet is lost, UDP itself does not resend it.

This is useful for applications where speed is more important than perfect delivery, such as:

- Live streaming
- Online gaming
- Voice communication
- Video conferencing

Comparison

Feature	TCP	UDP
Connection	Connection-oriented	Connectionless
Sequence numbers	Yes	No
ACK	Yes	No
Retransmission	Yes	No
Reliability	High	Lower
Speed	Relatively slower	Faster
Overhead	Higher	Lower

Why TCP is Slower

TCP requires additional processing for:

1. Acknowledgments

2. Sequence number checking
3. Retransmission
4. Flow control
5. Congestion control

Therefore, TCP generally has more overhead than UDP.

Result

TCP retransmission was observed using Wireshark. TCP retransmits lost packets using sequence numbers and acknowledgments, making it reliable. UDP does not automatically retransmit lost packets, which reduces overhead and makes it faster but less reliable.

Experiment 4: DNS Traffic and Resolution Time

Aim

To capture DNS traffic using Wireshark, identify DNS query types such as A, AAAA, and MX, observe the response, and measure the time required for domain-name resolution.

Requirements

- Computer/Laptop
- Wireshark
- Internet connection
- Command Prompt/Terminal

Procedure

1. Open Wireshark.
2. Select the active network interface.
3. Start packet capture.
4. Open Command Prompt.
5. Clear the local DNS cache if required.

For Windows:

`ipconfig /flushdns`

6. Perform a DNS lookup:

`nslookup google.com`

7. Stop the Wireshark capture.
8. Apply the filter:

`dns`

9. Select the DNS query packet.
10. Expand the Domain Name System section.
11. Observe the query type and response.

DNS Query Types

1. A Record

The A record maps a domain name to an IPv4 address.

Example:

`google.com` → IPv4 address

2. AAAA Record

The AAAA record maps a domain name to an IPv6 address.

Example:

google.com → IPv6 address

3. MX Record

The MX record identifies the mail servers responsible for receiving email for a domain.

Example:

example.com → Mail server

Wireshark Observation

In the DNS query packet, observe:

Queries

Name: google.com

Type: A

The response may contain:

Answers

Name: google.com

Type: A

Address: <IPv4 address>

The actual IP address will depend on the DNS server and time of capture.

Measuring Resolution Time

Wireshark displays a Time value for each packet.

Identify:

DNS Query



DNS Response

The approximate DNS resolution time can be calculated as:

Resolution Time = Response Time - Query Time

For example, if:

Query Time = 5.000000 s

Response Time = 5.025000 s

then:

Resolution Time = 5.025000 - 5.000000

= 0.025 seconds

= 25 ms

Your actual value will be different depending on network conditions and DNS server response time.

Why DNS Uses UDP

DNS commonly uses UDP port 53 because DNS queries are generally:

- Small
- Fast
- Simple

- Connectionless

Using UDP avoids the TCP three-way handshake.

The communication is approximately:

Client DNS Server

| ---- DNS Query -----> |

| <---- DNS Response ---- |

This provides low latency.

DNS can also use TCP, particularly when responses are large or for certain DNS operations. Modern DNS mechanisms can also use encrypted transports such as DNS over HTTPS or DNS over TLS.

Result

DNS traffic was successfully captured in Wireshark. The DNS query type, such as A, AAAA, or MX, and its corresponding response were identified. The DNS resolution time was calculated by comparing the query and response timestamps. DNS commonly uses UDP because it provides low overhead and fast communication.